Algorithmes et Pensée Computationnelle

Algorithmes de tri et Complexité - exercices basiques

Le but de cette série d'exercices est d'aborder les notions présentées durant la séance de cours. Cette série d'exercices sera orientée autour des points suivants :

1. La récursivité
2. La complexité des algorithmes
3. Les algorithmes de tri

Les exercices de cette séance seront réalisés en Java et Python.

Le temps indiqué (🕒) est à titre indicatif.

1 Récursivité

Le but principal de la récursivité est de résoudre un gros problème en le divisant en plusieurs petites parties à résoudre.

Question 1: (🕒 *10 minutes*) **Somme des chiffres**
Écrivez une fonction récursive en Python et en Java qui prend un entier en paramètre et retourne la somme des chiffres dont il est composé. Par exemple, la somme des chiffres de 126 est : 1+2+6 = 9.

Conseil

Pour obtenir le chiffre des unités d'un nombre, utilisez l'opérateur modulo `%`. Par exemple : $42 \bmod 10 = 2$, car le reste de la division entière de 42 par 10 est 2. Ensuite, pour obtenir le chiffre 4, effectuez une division entière par 10 : en Python, `42 // 10` vaut 4, en Java, `42 / 10` vaut 4, car les deux opérandes sont des entiers.

Solution

Python :

```python
def sum_digits(number):
    if number == 0:
        return 0
    else:
        return (number % 10) + sum_digits(number // 10)

print(sum_digits(126))
```

Java :

```java
public class SumDigits {
    public static int sumDigits(int number) {
        if (number == 0) {
            return 0;
        } else {
            return (number % 10) + sumDigits(number / 10);
        }
    }

    public static void main(String[] args) {
        System.out.println(sumDigits(126));
    }
}
```

2 Complexité

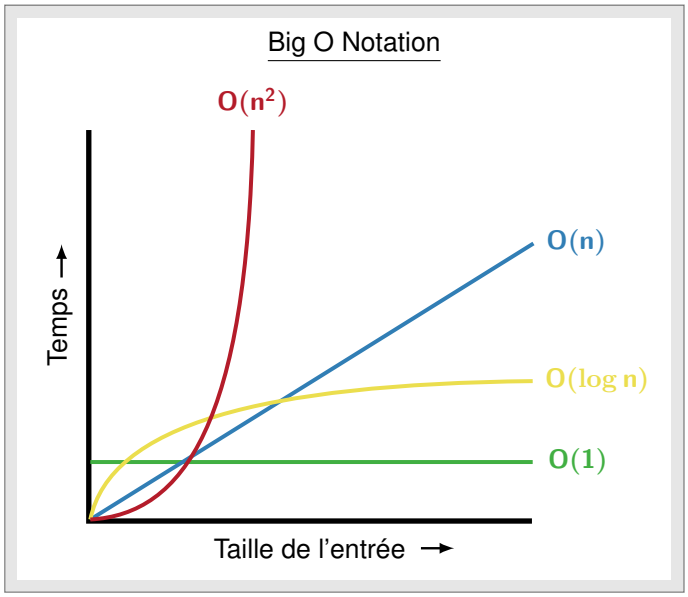


FIGURE 1 – Représentation de complexités temporelles. La complexité décrit la façon dont le temps d'exécution **croît asymptotiquement** lorsque la taille des données augmente, et non sa valeur exacte.

Indiquez en une phrase, ce que font ces algorithmes ci-dessous et calculez leur complexité temporelle avec la notation O. Le code est écrit en Python et en Java.

Question 2: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?
Python :

```python
# Entrée: n un nombre entier
def algo(n):
    s = 0
    for i in range(10*n):
        s += i
    return s
```

Java :

```java
public static int algo(int n) {
    int s = 0;
    for (int i=0; i < 10*n; i++){
        s += i;
    }
    return s;
}
```

(A) $O(n)$
(B) $O(n^3)$
(C) $O(\log(n))$
(D) $O(n^n)$

Conseil

Rappelez-vous que la notation O sert à exprimer la complexité d'algorithmes dans le **pire des cas**. Les règles suivantes vous seront utiles. Soit n la taille de vos données. On a alors :

1. Les constantes sont ignorées : $O(2n) = 2 \cdot O(n) = O(n)$
2. Les termes dominés sont ignorés : $O(2n^2 + 5n + 50) = O(n^2)$

Solution

L'algorithme est composé d'une boucle qui incrémente une variable `s`. Il effectue l'opération d'addition $10 \cdot n$ fois et par conséquent a une complexité de $O(n)$ comme les facteurs constants sont ignorés.

Question 3: (*10 minutes*) **Complexité**

Soit `values` une liste de longueur n. Quelle est la complexité des programmes ci-dessous ?

Python :

```python
# Entrée: values est une liste de nombres entiers et threshold un nombre entier
def algo(values, threshold):
    i = 0
    while i < len(values) and values[i] <= threshold:
        i += 1
    s = i - 1
    return s

```

Java :

```java
public static int algo(int[] values, int threshold) {
    int i = 0;
    while (i < values.length && values[i] <= threshold){
        i += 1;
    }
    int s = i - 1;
    return s;
}

```

(A) $O(n^3)$

(B) $O(\log(n))$

(C) $O(n)$

(D) $O(n^n)$

Solution

L'algorithme est composé d'une boucle `while` qui parcourt la liste `values` jusqu'à trouver une valeur supérieure à `threshold`. Ainsi, dans le pire des cas, l'algorithme parcourt toute la liste, et a donc une complexité de $O(n)$, où n est la taille de la liste.

Question 4: (*10 minutes*) **Complexité**

Quelle est la complexité des programmes ci-dessous ?

Python :

```python
# Entrée: first_values et second_values sont deux listes de nombres entiers
def algo(first_values, second_values):
    n = len(first_values)
    m = len(second_values)
    for i in range(n):
        first_values[i] = first_values[i]*2
```

```python
    for j in range(m):
        second_values[j] = second_values[j]%2

```

Java :

```java
public static void algo(int[] firstValues, int[] secondValues) {
    int n = firstValues.length;
    int m = secondValues.length;
    for (int i=0; i < n; i++){
        firstValues[i] = firstValues[i]*2;
    }
    for (int j=0; j < m; j++){
        secondValues[j] = secondValues[j]%2;
    }
}

```

(A) $O(n^2)$
(B) $O(n+m)$
(C) $O(n)$
(D) $O(2^n)$

Solution

L'algorithme est composé de deux boucles. La première parcourt la liste `first_values` en Python (`firstValues` en Java) et multiplie par 2 ses éléments. L'autre parcourt la liste `second_values` en Python (`secondValues` en Java) et affecte à chaque élément le reste de la division euclidienne de l'élément par 2. Soient n et m les tailles respectives de ces deux listes, on obtient une complexité de $O(n+m)$. Comme les boucles sont exécutées l'une après l'autre, leurs complexités s'additionnent.

Question 5: (*10 minutes*) **Complexité**
Quelle est la complexité des programmes ci-dessous ?
Python :

```python
# Entrée: values est une liste de nombres entiers
import math
def algo(values):
    n = len(values)
    i = 1
    s = 0
    while i < n:
        s += values[i]
        i = 2*i
    return s

```

Java :

```java
import java.lang.Math;

public static int algo(int[] values) {
    int n = values.length;
    int i = 1;
    int s = 0;
    while (i < n){
        s += values[i];
        i = 2*i;
    }
    return s;
}
```

(A) $O(n^2)$

(B) $O(n)$

(C) $O(\log(n))$

(D) $O(n^n)$

Solution

L'algorithme consiste en une boucle qui ajoute à la variable s les valeurs de la liste `values`, à condition que les indices de ces valeurs soient des puissances de 2 inférieures à n. La variable `i` prend successivement les valeurs $1, 2, 4, \ldots, 2^{\lfloor \log_2(n) \rfloor}$.

Pour montrer que $2^{\lfloor \log_2(n) \rfloor}$ est la plus grande puissance de 2 inférieure ou égale à n, remarquons que $n = 2^{\log_2(n)}$ et que $2^{\lfloor \log_2(n) \rfloor} \leq n < 2^{\lfloor \log_2(n) \rfloor + 1}$. Ainsi, n est encadré par deux puissances de 2 consécutives.

En réécrivant la suite des valeurs de `i` sous la forme $2^0, 2^1, \ldots, 2^{\lfloor \log_2(n) \rfloor}$, on constate que la boucle effectue $\lfloor \log_2(n) \rfloor$ itérations, d'où une complexité en $O(\lfloor \log_2(n) \rfloor) = O(\log(n))$, puisque la notation O décrit une croissance asymptotique et non des valeurs exactes.

Question 6: (*10 minutes*) **Complexité** Optionnel

Quelle est la complexité du programme ci-dessous ?

Python :

```python
# Entrée: n un nombre entier
def algo(n):
    m = 0
    for i in range(n):
        for j in range(i):
            m += i+j
    return m

```

Java :

```java
public static int algo(int n) {
    int m = 0;
    for (int i=0; i < n; i++){
        for (int j=0; j < i; j++){
            m += i+j;
        }
    }
    return m;
}

```

(A) $O(n^2)$

(B) $O(n)$

(C) $O(\log(n))$

(D) $O(2^n)$

Solution

L'algorithme est composé de deux boucles imbriquées. La boucle extérieure est exécutée n fois. Le nombre d'itérations de la boucle intérieure dépend de l'itération courante de la boucle extérieure : pour une valeur donnée de `i`, la boucle intérieure est exécutée `i` fois. Comme `i` prend successivement les valeurs $0, 1, \ldots, n-1$, le nombre total d'exécutions de l'instruction `m +=`

`i+j` est :

$$0+1+2+\cdots+(n-1)=\sum_{i=0}^{n-1} i=\frac{n(n-1)}{2}=\frac{n^2}{2}-\frac{n}{2}.$$

Chaque exécution de cette instruction prend un temps constant. On ne conserve que le terme quadratique, car il définit le comportement asymptotique de la fonction : les termes dominés et les facteurs constants sont ignorés. La complexité temporelle est donc $O(n^2)$.

3 Algorithmes de Tri

Question 7: (*15 minutes*) **Insertion dans une liste triée - Python**
Soit un nombre entier `n`, et une liste triée `l`. Ecrivez un programme Python qui insère la valeur `n` dans la liste `l` tout en s'assurant que la liste `l` reste triée.

```
1 def insertion_entier(liste, number):
2     # TODO : Compléter ici
3
4 print(insertion_entier([2, 4, 6], 1))
```

Exemple

En passant les arguments suivants à votre programme : `n=5` et `l=[2,4,6]`. Ce dernier devra retourner `l =[2,4,5,6]`

Solution

```
1  def insertion_entier(liste, number):
2      # ajoute un élément à la liste
3      liste.append(number)
4      n = len(liste) - 1
5      while n > 0 and liste[n - 1] > number:
6          liste[n] = liste[n - 1]
7          n -= 1
8      liste[n] = number
9      return liste
10
11 # solution alternative
12 def insertion_entier(liste,number):
13     for i in range(len(liste)):
14         if number <= liste[i] :
15             return liste[:i] + [number] + liste[i:]
16     return liste + [number]
17
18 print(insertion_entier([2, 4, 6], 1))
```

Question 8: (*30 minutes*) **Tri fusion (Merge Sort) - Python**
À partir de deux listes triées, on peut facilement construire une liste triée comportant les éléments issus de ces deux listes (leur *fusion*). Le principe de l'algorithme de tri fusion repose sur cette observation : le plus petit élément de la liste à construire est soit le plus petit élément de la première liste, soit le plus petit élément de la deuxième liste. Ainsi, on peut construire la liste élément par élément en retirant tantôt le premier élément de la première liste, tantôt le premier élément de la deuxième liste (en fait, le plus petit des deux, à supposer qu'aucune des deux listes ne soit vide, sinon la réponse est immédiate).
Les étapes à suivre pour implémenter l'algorithme sont les suivantes :

1. Si le tableau n'a qu'un élément, il est déjà trié.

2. Sinon, séparer le tableau en deux parties plus ou moins égales.
3. Trier récursivement les deux parties avec l'algorithme de tri fusion.
4. Fusionner les deux tableaux triés en un seul tableau trié.

Soit la liste `l` suivante [38, 27, 43, 3, 9, 82, 10], triez les éléments de la liste en utilisant un tri fusion. Combien d'itération effectuez-vous ?

- **Python :**

```python
def merge(partie_gauche, partie_droite):
    # TODO: Code à compléter

def tri_fusion(liste):
    # TODO: Code à compléter

if __name__ == "__main__":
    l = [38, 27, 43, 3, 9, 82, 10]
    print(tri_fusion(l))
```

Conseil

- L'algorithme est récursif.
- Revenez à la visualisation de l'algorithme dans les diapositives 81 à 109 pour comprendre comment fonctionne concrètement le tri fusion.

Conseil

Théorème 1 La récurrence

$$T(n) = a\,T(n/b) + cn^k$$
$$T(1) = c$$

où a, b, c et k sont des constantes, se résout comme suit :

$$T(n) \in O(n^k) \quad \text{si } a < b^k$$
$$T(n) \in O(n^k \log n) \quad \text{si } a = b^k$$
$$T(n) \in O(n^{\log_b a}) \quad \text{si } a > b^k$$

Solution

Python :

```python
def merge(partie_gauche, partie_droite):
    # créer la liste qui sera retournée à la fin
    liste_fusionnee = []

    # définir un compteur pour l'index de la liste de gauche
    compteur_gauche = 0
    # pareil pour la liste de droite
    compteur_droite = 0

    longueur_gauche = len(partie_gauche)
    longueur_droite = len(partie_droite)

    # continuer jusqu'à ce que l'un des index (ou les deux) atteigne l'une des longueurs (ou les deux)
    while compteur_gauche < longueur_gauche and compteur_droite < longueur_droite:
        # comparer les éléments actuels, ajouter le plus petit à la
```

```
        liste fusionnée
16          # et augmenter le compteur de cette liste
17          if partie_gauche[compteur_gauche] <
        partie_droite[compteur_droite]:
18              liste_fusionnee.append(partie_gauche[compteur_gauche])
19              compteur_gauche += 1
20          else:
21              liste_fusionnee.append(partie_droite[compteur_droite])
22              compteur_droite += 1
23
24      # s'il y a encore des éléments dans les listes, il faut les
        ajouter à la liste fusionnée
25      liste_fusionnee += partie_gauche[compteur_gauche:longueur_gauche]
26      liste_fusionnee += partie_droite[compteur_droite:longueur_droite]
27
28      return liste_fusionnee  # retourner la liste fusionnée
```

Solution

```
31  def tri_fusion(l):
32      # compléter la fonction
33      longueur = len(l)  # calculer la longueur de la liste
34      # s'il n'y a pas plus d'un élément, retourner la liste
35      if longueur == 1 or longueur == 0:
36          return l
37      # sinon, diviser la liste en deux
38      elif longueur > 1:
39          # convertir la variable en nombre entier (l'index ne peut pas ê
        tre un nombre à virgule)
40          index_milieu = int(longueur / 2)
41          # la partie gauche va du 1er élément à celui du milieu
42          partie_gauche = l[0:index_milieu]
43          # la partie droite va du milieu à la fin de la liste
44          partie_droite = l[index_milieu:longueur]
45
46          # appeler la fonction tri_fusion à nouveau sur la partie
        gauche (récursivité)
47          partie_gauche_triee = tri_fusion(partie_gauche)
48          # même chose pour la partie droite
49          partie_droite_triee = tri_fusion(partie_droite)
50
51          liste_fusionnee = merge(partie_gauche_triee,
        partie_droite_triee)  # enfin, joindre les 2 parties
52
53          # retourner le résultat
54          return liste_fusionnee
55
56
57  if __name__ == "__main__":
58      l = [38, 27, 43, 3, 9, 82, 10]
59      print(tri_fusion(l))
```

Solution

Le tri fusion est un algorithme récursif. Ainsi, nous pouvons exprimer sa complexité temporelle via une relation de récurrence : $T(n) = 2T(n/2) + O(n)$. En effet, l'algorithme comporte 3 étapes :

1. "Divide Step", qui divise les listes en deux sous-listes, et cela prend un temps constant

2. “Conquer Step”, qui trie récursivement les sous-listes de taille $n/2$ chacune, et cette étape est représentée par le terme $2T(n/2)$ dans l'équation.
3. La dernière étape consiste à fusionner les listes, sa complexité est de $O(n)$.

La solution à cette équation est $O(n \log n)$.

Question 9: (*20 minutes*) **Tri à bulles (Bubble Sort) - Python** **Optionnel**

Le tri à bulles consiste à parcourir une liste et à comparer ses éléments. Le tri est effectué en permutant les éléments de telle sorte que les éléments les plus grands soient placés à la fin de la liste.
Concrètement, si un premier nombre x est plus grand qu'un deuxième nombre y et que l'on souhaite trier l'ensemble par ordre croissant, alors x et y sont mal placés et il faut les inverser. Si, au contraire, x est plus petit que y, alors on ne fait rien et l'on compare y à z, l'élément suivant.
Soit la liste `l = [1, 2, 4, 3, 1]`, triez les éléments de la liste en utilisant un tri à bulles. Combien d'itérations effectuez-vous ?

- **Python :**

```python
def tri_bulle(l):
    for i in range(len(l)):
        #TODO: Code à compléter

if __name__ == "__main__":
    l = [1, 2, 4, 3, 1]
    tri_bulle(l)
    print(l)
```

Solution

Python :

```python
def tri_bulle(l):
    n = len(l)

    for i in range(n):
        # Les i derniers éléments sont dans leur bonne position
        for j in range(0, n-i-1):

            # parcourir la liste de 0 à n-i-1
            # Echanger si l'élément trouvé est supérieur
            # au prochain élément
            if l[j] > l[j+1] :
                l[j], l[j+1] = l[j+1], l[j]

if __name__ == "__main__":
    l = [1, 2, 4, 3, 1]
    tri_bulle(l)
    print(l)
```

Solution

L'algorithme a une complexité de $O(n^2)$ car il contient deux boucles qui parcourent la liste.